EX NO: 12
08.10.26

12. a) IMPLEMENT ECHO CLIENT SERVER USING TCP/UDP SOCKETS

Aim:

To implement end-to-end communication at the transport layer using TCP/UDP socket programming and demonstrate echo-communication between client and server.

TCP echo code:

server:

```
import socket
import threading
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1',5000))
    server.listen(1)
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] connection closed by client")
            break
        print(f"[SERVER] Received: {data}")
        conn.send(f"server received: {data}".encode())
    conn.close()
    server.close()
```

client:

```
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")
```

```
        while True:
            msg = input("Enter message (or 'quit to exit:")
            client.send(msg.encode())
            if msg.lower() == 'quit':
                break
            response = client.recv(1024).decode()
            print(f"[SERVER RESPONSE] {response}")
        client.close()
if __name__ == "__main__":
        server_thread = threading.Thread(
            target = start_server,
            daemon = True
        )
        server_thread.start()
        start_client()
```

o/p:

```
[SERVER] Listening on 127.0.0.1:5000
[CLIENT] Connected to sever
Enter message (or 'quit' to exit): [SERVER] connected by
                ('127.0.0.1', 47922)
Hello
[SERVER] Received: Hello
[SERVER RESPONSE] Server received: Hello
```

UDP echo program:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
```

```
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] client {addr} ended the connection")
            break
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] stopped")
def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] closing connection")
            break
        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.decode())
    c.close()
print("Echo from server:", data.decode())
print("[CLIENT] stopped")
if __name__ == "__main__":
    threading.Thread(
        target = udp_server,
        daemon = True
    ).start()
    udp_client()
```

O/P:

Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000

udp

[SERVER] Received from('127.0.0.1', 54425): udp

Echo from server: udp

E

Result:

Thus, end to end communication b/w client and server was successfully implemented using TCP and UDP socket programming.

12. b) IMPLEMENT CHAT CLIENT SERVER USING TCP/UDP SOCKETS.

Aim:

To implement a chat program using socket programming for communication between a client and a server.

chat program:

```
import socket
import threading
def receive_messages(conn):
    while True:
        try:
            msg = conn.recv(1024).decode()
            if not msg or msg.lower() == "quit":
                print("\n[CLIENT DISCONNECTED]")
                break
            print("\nClient: ", msg)
        except:
            break
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind('127.0.0.1', 5001))
    server.listen (1)
    print("[SERVER] Waiting for client...")
    conn, addr = server.accept()
    print("[SERVER] client connected!")
    threading.thread(
        target = receive_messages,
        args = (conn,),
        daemon = True
    ).start()
    while True:
        msg = input("Server: ")
        conn.send(msg.encode())
```

```
        if msg.lower() == "quit":
            break
    conn.close()
    server.close()
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STR
    client.connect(('127.0.0.1', 5001))
    print("[CLIENT] connected to server!")
    threading.Thread(
        target = received_messages,
        args = (client,),
        daemon = True
    ).start()
    while True:
        msg = input("client: ")
        client.send(msg.encode())
        if msg.lower() == "quit":
            break
    client.close()
choice = input("Enter S for server or C for client:")
if choice.upper() == "S":
    start_server()
elif choice.upper() == "C":
    start_client()
```

O/P: (client side)

Enter S for server or C for client: C

[CLIENT] connected to server!

client:

client: Hello

Hii Server

client: quit.

Enter S for server or C for client : S
[SERVER] Waiting for client...
[SERVER] Client connected!
Server : Hello
Server :
Client : Hei Server
[CLIENT DISCONNECTED]

Result : Ok

Thus, the chat program was successfully implemented using socket programming.